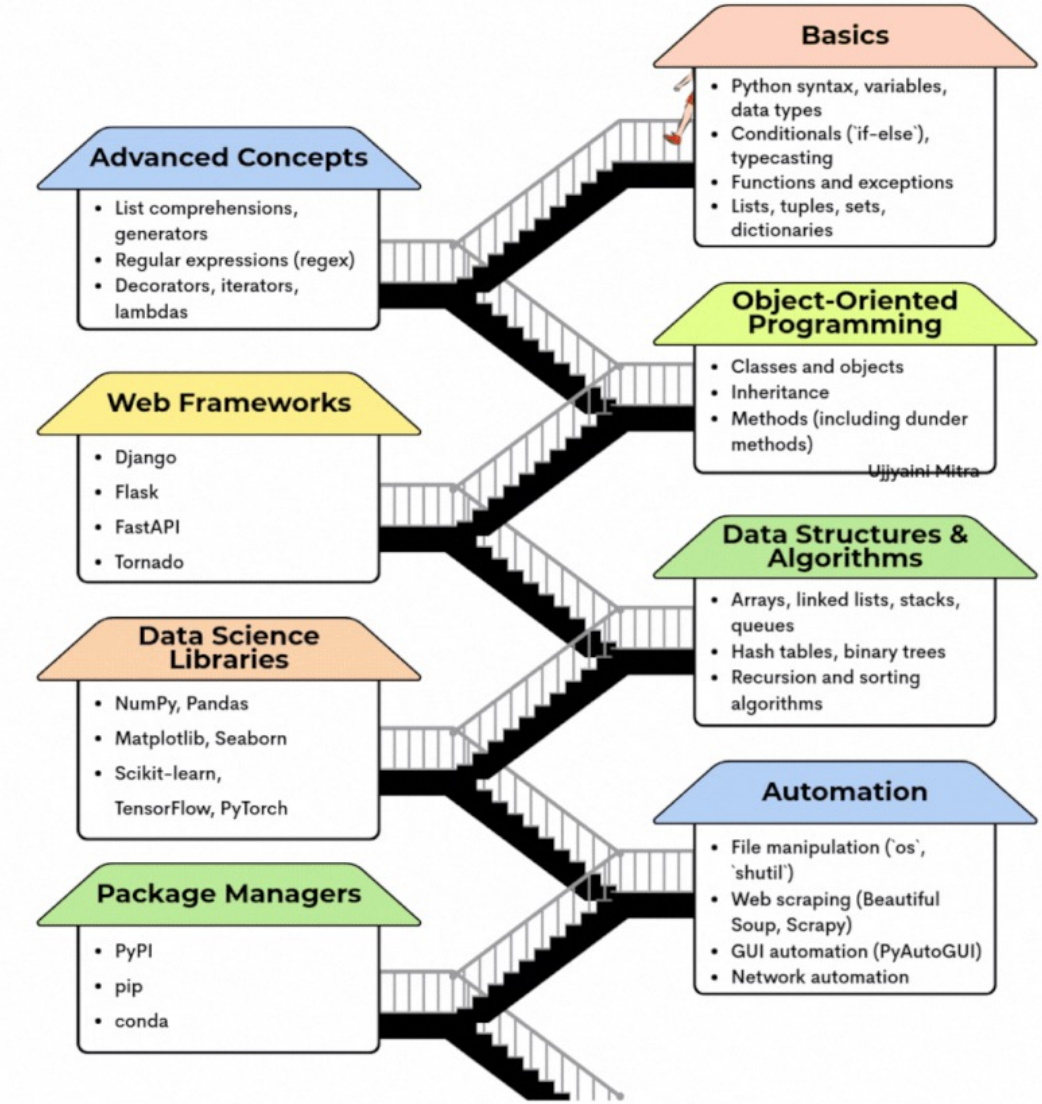
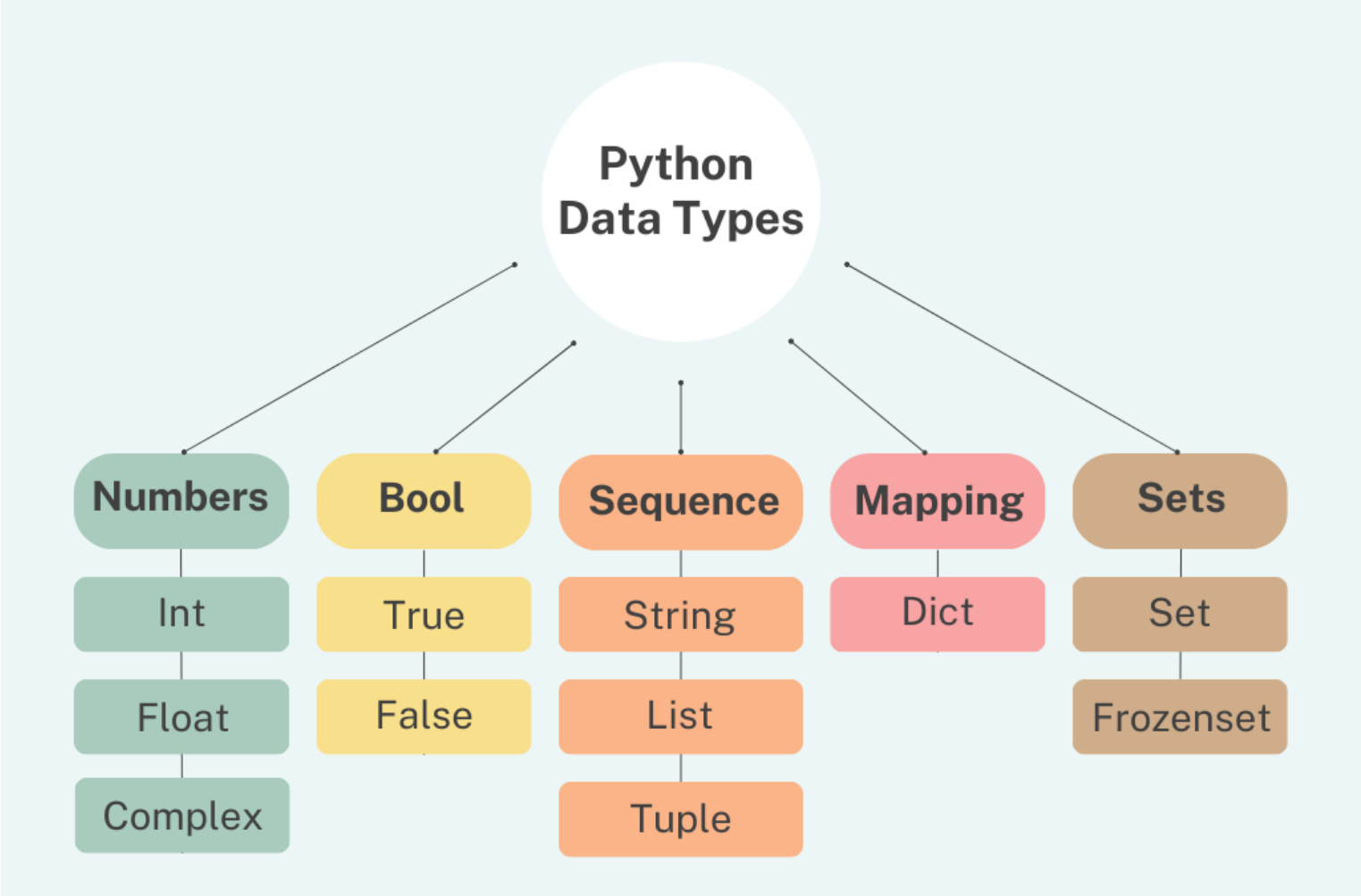
INTRODUCTION TO PYTHON

# Lists, Ranges & Tuples Dictionaries & Sets Input/Output & Built-in Functions

Core data structures and everyday tools every Python program relies on


Basics
Python syntax, variables, data types
Conditionals (`if-else`), typecasting
Functions and exceptions
Lists, tuples, sets, dictionaries
Advanced Concepts
List comprehensions, generators
Regular expressions (regex)
Decorators, iterators, lambdas
Object-Oriented Programming
Classes and objects
Inheritance
Methods (including dunder methods)
Ujjyaini Mitra
Web Frameworks
Django
Flask
FastAPI
Tornado
Data Structures & Algorithms
Arrays, linked lists, stacks, queues
Hash tables, binary trees
Recursion and sorting algorithms
Data Science Libraries
NumPy, Pandas
Matplotlib, Seaborn
Scikit-learn, TensorFlow, PyTorch
Automation
File manipulation (`os`, `shutil`)
Web scraping (Beautiful Soup, Scrapy)
GUI automation (PyAutoGUI)
Network automation
Package Managers
PyPI
pip
conda


Python
Data Types
Numbers
Int
Float
Complex
Bool
True
False
Sequence
String
List
Tuple
Mapping
Dict
Sets
Set
Frozenset

# LIST VS TUPLE VS SET VS DICTIONARY
## in Python

@pycode.hubb

| FEATURE | LIST | TUPLE | SET | DICTIONARY |
|---|---|---|---|---|
| Syntax | [1, 2, 3] | (1, 2, 3) | {1, 2, 3} | {"a": 1, "b": 2} |
| Ordered? | ✅ Yes | ✅ Yes | ❌ No | ✅ Yes |
| Mutable? | ✅ Yes | ❌ No | ✅ Yes | ✅ Yes |
| Duplicates? | ✅ Allowed | ✅ Allowed | ❌ Not Allowed | ✅ Keys unique |
| Use Case | General collection | Fixed data | Unique items | Key-value mapping |

| List | Tuple | Set | Dictionary |
|---|---|---|---|
| A list is a non-homogeneous data structure that stores the elements in columns of a single row or multiple rows. | A Tuple is a non-homogeneous data structure that stores elements in columns of a single row or multiple rows. | The set data structure is non-homogeneous but stores the elements in a single row. | A dictionary is also a non-homogeneous data structure that stores key-value pairs. |
| The list can be represented by [ ] | A tuple can be represented by ( ) | The set can be represented by { } | The dictionary can be represented by { } |
| The list allows duplicate elements | Tuple allows duplicate elements | The Set will not allow duplicate elements | The dictionary doesn't allow duplicate keys. |
| The list can be nested among all | A tuple can be nested among all | The set can be nested among all | The dictionary can be nested among all |
| Example: [1, 2, 3, 4, 5] | Example: (1, 2, 3, 4, 5) | Example: {1, 2, 3, 4, 5} | Example: {1: "a", 2: "b", 3: "c", 4: "d", 5: "e"} |

| List | Tuple | Set | Dictionary |
|---|---|---|---|
| A list can be created using the **list()** function | Tuple can be created using the **tuple()** function. | A set can be created using the **set()** function | A dictionary can be created using the **dict()** function. |
| A list is mutable i.e we can make any changes in the list. | A tuple is immutable i.e we can not make any changes in the tuple. | A set is mutable i.e we can make any changes in the set, its elements are not duplicated. | A dictionary is mutable, its Keys are not duplicated. |
| List is ordered | Tuple is ordered | Set is unordered | Dictionary is ordered (Python 3.7 and above) |
| Creating an empty list<br>l=[] | Creating an empty Tuple<br>t=() | Creating a set<br>a=set()<br>b=set(a) | Creating an empty dictionary<br>d={} |

01

# Lists, Ranges & Tuples

➔ Ordered collections that store multiple values

➔ One mutable, one immutable — same indexing rules

UNIT — DATA STRUCTURES

# Where Lists & Tuples Fit In

➔Python's built-in data structures organize data for us: List, Tuple, Dictionary, Set

➔A List is an ordered, mutable sequence — created with square brackets [ ]

➔A Tuple is an ordered, immutable sequence — created with parentheses ( )

➔range() generates a sequence of numbers, most often used to drive loops and indexing

➔All three are iterable — we can loop over them, index into them, and slice them

*Reference: Allen B. Downey, Think Python 2e — Ch. 10 (Lists) & Ch. 12 (Tuples)*

# What is a List?

```
a = [12, 'abc', 8.95, [2, 6]]
```

A list is created using square brackets [ ]. Lists are mutable — elements can be added, removed, or changed after creation. A single list can hold multiple elements of the same or different data types, including other lists (nesting).

```
>>> a = []          # empty list
>>> b = [4, 7, 2, 4]
```

# List Basics — Concatenation & Membership

Lists support +, *, in, and the min/max/sum built-ins (numeric lists).

```
a = [12, 'abc', 8.95, "xyz", [2, 6]]
b = [4, 7, 2, 4]

c = a + b                  # concatenation
print("Length of c:", len(c))
print("'abc' in a:", 'abc' in a)
print("2 * b:", 2 * b)     # repeats the list
print("sum(b):", sum(b))   # numeric lists only
```

**OUTPUT**

```
Length of c: 9

'abc' in a: True

2 * b: [4, 7, 2, 4,
        4, 7, 2, 4]

sum(b): 17
```

# List Functions — Growing & Shrinking a List

| Function | What it does | Example |
|---|---|---|
| append(x) | Adds x to the end of the list | a.append(6) |
| extend([x,y]) | Adds every element of another list | a.extend([7, 8, 9]) |
| insert(i, x) | Inserts x at index position i | a.insert(4, 'Text') |
| pop() | Removes and returns the last element | a.pop() |
| pop(i) | Removes and returns element at index i | a.pop(2) |
| del a[i] | Deletes element at index i | del a[0] |

# Accessing & Modifying List Elements

Indexing starts from 0. Lists are mutable, so a[i] can be reassigned directly.

```
a = [1, 2, 3, 4, 5]
L = len(a)

print("a[2]:", a[2])        # 3rd element
print("a[0]:", a[0])        # 1st element
print("a[L-1]:", a[L - 1])  # last element

a[2] = 6                    # 3 -> 6 (mutable!)
print("After a[2]=6:", a)
```

**OUTPUT**

```
a[2]: 3
a[0]: 1
a[L-1]: 5

After a[2]=6:
[1, 2, 6, 4, 5]
```

UNIT — DATA STRUCTURES

# Slicing & Nested Lists

a[start:stop] extracts a sub-list. Nested lists are accessed with chained indices.

```
a = [1, 5, 8, 2, 5, 2]
print("a[3:5]:", a[3:5])   # index 3 to 4
print("a[1:]:", a[1:])     # index 1 to end
print("a[:3]:", a[:3])     # start to index 2

b = [12, 'abc', ["Text", 5.34], "xyz"]
print("b[2]:", b[2])
print("b[2][1]:", b[2][1])  # nested access
```

**OUTPUT**

```
a[3:5]: [2, 5]
a[1:]: [5, 8, 2, 5, 2]
a[:3]: [1, 5, 8]

b[2]: ['Text', 5.34]
b[2][1]: 5.34
```

# More List Functions

| Function | What it does | Example |
|---|---|---|
| count(x) | Counts occurrences of x | a.count(5) |
| index(x) | Returns the first index of x | a.index(5) |
| remove(x) | Removes the first occurrence of x | a.remove(5) |
| reverse() | Reverses the list in place | a.reverse() |
| sort() | Sorts ascending in place | a.sort() |
| sort(reverse=True) | Sorts descending in place | a.sort(reverse=True) |
| sort(key=fn) | Sorts using a key function | words.sort(key=len) |

*sort(key=len) on a list of words sorts them shortest-to-longest instead of A-Z.*

# The range() Function

```
range(start, stop, step)
```

range() generates a sequence of numbers from start (default 0) up to, but NOT including, stop, advancing by step (default 1). It's memory-efficient — values are produced on demand, not stored as a list. Wrap it in list() to see all the values at once.

```
>>> list(range(5))          -> [0, 1, 2, 3, 4]
>>> list(range(4, 8))       -> [4, 5, 6, 7]
>>> list(range(1, 10, 2))   -> [1, 3, 5, 7, 9]
```

# range() in Practice

range() is most often used to control loops, or to generate index positions for a list.

```
print(range(5), list(range(5)))
print(range(4, 8), list(range(4, 8)))
print(range(1, 10, 2), list(range(1, 10, 2)))

a = [5, 2, 6, 2, 5]
for i in range(len(a)):
    print("Element", i, ":", a[i])
```

**OUTPUT**

```
range(0, 5) [0, 1, 2, 3, 4]
range(4, 8) [4, 5, 6, 7]
range(1, 10, 2) [1, 3, 5, 7,
9]

Element 0 : 5
Element 1 : 2
Element 2 : 6  ...
```

# Lists vs. Tuples

## List [ ]

- Mutable — can change after creation
- Defined with square brackets [ ]
- Supports append, insert, remove, sort, reverse
- Slightly more memory & processing overhead
- Best for data that changes over time

## Tuple ( )

- Immutable — cannot change after creation
- Defined with parentheses ( )
- reverse(), remove(), sort() do NOT work
- Lighter weight and slightly faster
- Best for fixed data — coordinates, records

# Tuple Basics

Tuples support the same read-only operations as lists: length, sum, count, index.

```
a = (1, 5, 8, 2, 5, 2)
print("a:", a)
print("Length:", len(a))
print("Sum:", sum(a))
print("a.count(5):", a.count(5))
print("a.index(8):", a.index(8))
```

**OUTPUT**

```
a: (1, 5, 8, 2, 5, 2)
Length: 6
Sum: 23
a.count(5): 2
a.index(8): 2
```

UNIT — DATA STRUCTURES

# sorted() and reversed() — Work on Any Iterable

Unlike a.sort(), sorted(a) returns a NEW list and leaves the tuple untouched.

```
a = (1, 5, 8, 2, 5, 2)

b = sorted(a)                  # returns a list
print("sorted(a):", b, type(b))

c = sorted(a, reverse=True)
print("descending:", c)

d = list(reversed(a))          # reversed() returns an
iterator
print("reversed(a):", d)
```

OUTPUT

```
sorted(a): [1, 2, 2, 5, 5, 8]
<class 'list'>

descending:
[8, 5, 5, 2, 2, 1]

reversed(a): [2, 5, 2, 8, 5, 1]
```

# Defining Tuples — the Tricky Cases

A single-element tuple NEEDS a trailing comma, or Python just sees plain parentheses.

```
a = (100,)              # trailing comma -> tuple!
print(a, type(a))

b = 1, 2, 3, 4, 5       # parentheses optional
print(b, type(b))

x, y = 10, 20           # tuple assignment
x, y = y, x             # swap in one line!
print(x, y)
```

OUTPUT

```
(100,) <class 'tuple'>

(1, 2, 3, 4, 5) <class 'tuple'>

20 10
```

UNIT — DATA STRUCTURES

# Tuple Unpacking

A method that returns multiple values is really returning one tuple — unpack it directly into named variables.

```
a = 'Jack-Martin'
first, sep, last = a.partition('-')
print(f'After removing {sep}: {first} {last}')

# use _ to ignore a value you don't need
b = 'Jack|24'
name, _, age = b.partition('|')
print(f'Name: {name}, Age: {age}')
```

OUTPUT

```
After removing -: Jack Martin

Name: Jack, Age: 24
```

02

# Python Dictionaries & Sets

➔ Dictionaries store key-value pairs

➔ Sets store unique, unordered elements

# What is a Dictionary?

```
d = {1: 'One', 2: 'Two', 3: 'Three'}
```

A dictionary stores data as key-value pairs inside curly braces { }. Keys must be unique and immutable (numbers, strings, tuples - never lists). Accessing or modifying a key that doesn't exist raises a KeyError.

```
>>> empty_dict = {}
>>> d1[2]  ->  'Two'
```

# Merging Dictionaries

d1.update(d2) merges in place. Python 3.9+ also offers the | operator and 3.5+ offers ** unpacking.

```
d1 = {0: "Zero", 1: "One", 2: "Two"}
d2 = {2: "two", 3: "Three"}

d1.update(d2)                # in-place merge
print("update():", d1)
```

OUTPUT

```
update(): {0: 'Zero', 1: 'One',
           2: 'two', 3: 'Three'}
```

# Merging Dictionaries

```
student = {"name": "Alice", "age": 20}
marks = {"python": 95, "math": 90}

result = student | marks

print(result)
```

OUTPUT

```
{'name': 'Alice', 'age': 20, 'python': 95, 'math': 90}
```

# If keys are duplicated

```
d1 = {"a": 10, "b": 20}
d2 = {"b": 50, "c": 30}

print(d1 | d2)
```

OUTPUT

```
{'a': 10, 'b': 50, 'c': 30}
```

The value from the **right-hand dictionary** (`d2`) overwrites the value from the left-hand dictionary (`d1`) for duplicate keys.

## ** (Dictionary Unpacking)

The ** operator unpacks the key-value pairs from one or more dictionaries into a new dictionary.

```
student = {"name": "Alice", "age": 20}
marks = {"python": 95, "math": 90}

result = {**student, **marks}

print(result)
```

OUTPUT

```
{'name': 'Alice', 'age': 20, 'python': 95, 'math': 90}
```

# Duplicate keys

The ** operator unpacks the key-value pairs from one or more dictionaries into a new dictionary.

```
d1 = {"a": 10, "b": 20}
d2 = {"b": 50, "c": 30}

result = {**d1, **d2}
print(result)
```

OUTPUT

```
{'a': 10, 'b': 50, 'c': 30}
```

The value from the **right-hand dictionary** (`d2`) overwrites the value from the left-hand dictionary (`d1`) for duplicate keys.

# What Can Be a Key or a Value?

➔ A key must be immutable: Boolean, Integer, Float, String, or Tuple - never a List

➔ A value can be ANY type - including mutable types like List or another Dictionary

➔ If the same key is written twice, the later value silently overwrites the earlier one

*a = {"Text": "String", 18: "Integer", True: "Boolean", (1,2,3): "Tuple"}*

➔ New keys can be added simply by assignment: a['New Key'] = 'New Value'

➔ Dictionaries do NOT preserve a guaranteed print/sort order - treat printed order as incidental

# Reading a Dictionary - keys(), values(), items()

These three methods are the standard way to inspect or loop through a dictionary.

```
b = {
    "Name": "Jack", "Age": 34,
    "Access": False, "Mobile": [12345, 95748]
}
print("b.keys():", b.keys())
print("b.values():", b.values())
print("b.items():", b.items())
```

OUTPUT

```
b.keys():
dict_keys(['Name','Age','Access','Mobil
e'])

b.values():
dict_values(['Jack',34,False,[...]])

b.items():
dict_items([('Name','Jack'), ...])
```

UNIT — DATA STRUCTURES

# Converting & Pretty-Printing Dictionaries

dict() can build a dictionary from a sequence of (key, value) tuples. pprint gives a clean, sorted, multi-line view.

```
x = (('a', 1), ('b', 2), ('c', 3))
y = dict(x)
print("Tuple -> dict:", y)

from pprint import pprint as pp
a = {"Name": "Jack", "Age": 34, "Access": False}
pp(a)
```

OUTPUT

```
Tuple -> dict:
{'a': 1, 'b': 2, 'c': 3}

{'Access': False,
 'Age': 34,
 'Name': 'Jack'}
```

# What is a Set?

```
s = {1, 2, 3, 4, 5}
```

A set is an unordered collection of UNIQUE elements, created with curly braces { } or the set() constructor. Duplicate values are automatically dropped. Sets are ideal for membership tests and mathematical set operations.

```
>>> {2,5,1,5,2}  ->  {1, 2, 5}   (duplicates removed)
```

# Set Basics - add, remove, discard, copy

remove() raises an error if the element is missing; discard() never does.

```
s1 = {1, 2, 3, 4, 5}
s1.add(6)                # add an element
s1.remove(6)              # error if missing
s1.discard(4)              # safe, no error either way
print("6 in s1:", 6 in s1)

s2 = s1.copy()
s2.clear()                  # empty the copy
print("s1:", s1, " s2:", s2)
```

OUTPUT

```
6 in s1: False

s1: {1, 2, 3, 5}
s2: set()
```

# Set Operations - Math You Already Know

| Operation | Method | Operator |
|---|---|---|
| Union (A U B) | a.union(b) | a \| b |
| Intersection (A ^ B) | a.intersection(b) | a & b |
| Difference (A - B) | a.difference(b) | a - b |
| Symmetric Difference | a.symmetric_difference(b) | a ^ b |
| Is Superset of C? | a.issuperset(c) | - |
| Is Subset of C? | c.issubset(a) | - |
| Disjoint (no overlap)? | c.isdisjoint(a) | - |

*For a = {1,2,3,4}, b = {3,4,5,6}: a \| b = {1,2,3,4,5,6}, a & b = {3,4}*

# Set Operations in Action

Given a = {1,2,3,4} and b = {3,4,5,6}:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print("Union a|b:", a | b)
print("Intersection a&b:", a & b)
print("Difference a-b:", a - b)
print("Symmetric a^b:", a ^ b)
```

**OUTPUT**

```
Union a|b: {1,2,3,4,5,6}

Intersection a&b: {3, 4}

Difference a-b: {1, 2}

Symmetric a^b: {1,2,5,6}
```

# Sets vs. Frozensets

## set { }

- Mutable - add(), remove(), discard(), update()
- Created with { } or set()
- Cannot be used as a dictionary key (unhashable)
- Good for data you'll keep modifying

## frozenset( )

- Immutable - no add(), no remove()
- Created with frozenset(iterable)
- CAN be used as a dictionary key or set element
- Signals to future readers: 'do not modify this'

# Frozensets

frozenset() wraps any iterable - list, set, or even a string - into an immutable set.

```
scores = {1, 2}
frozen_scores = frozenset(scores)

frozen_scores.add(3)
# AttributeError: 'frozenset' object
# has no attribute 'add'

myString = "Hello"
print(frozenset(myString))
```

OUTPUT

```
AttributeError:
'frozenset' object has
no attribute 'add'

frozenset({'e','l','H','o'})
```

03

# Input and Output in Python

- ➔ Reading from the keyboard with input()
- ➔ Formatting output with f-strings
- ➔ Reading and writing files

INPUT & OUTPUT

# Two Kinds of I/O We Use in Python

- ➔ Console I/O — talking to the user: input() reads text, print() displays it
- ➔ Formatted Output — controlling exactly how values appear using f-strings or .format()
- ➔ File I/O — reading from and writing to files on disk that outlive the program
- ➔ In every case: input() always returns a string — convert it explicitly when you need a number

INPUT & OUTPUT

# input() — Reading From the Keyboard

input() pauses the program, shows the prompt, and returns whatever the user typed — always as a string.

```
a = input("Enter number: ")
print("User entered:", a)
print(type(a))     # <class 'str'>, always!

b = int(a)          # convert string -> int
print("After conversion:", b)
print(type(b))     # <class 'int'>
```

OUTPUT

```
Enter number: 42
User entered: 42
<class 'str'>

After conversion: 42
<class 'int'>
```

INPUT & OUTPUT

# input() in a Real Program

Wrapping input() directly in int() or float() is the standard one-line conversion pattern.

```
marks = int(input("Enter Marks: "))
print("Marks scored:", marks)

if marks >= 75:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

OUTPUT

```
Enter Marks: 82
Marks scored: 82
Grade A
```

INPUT & OUTPUT

# Three Ways to Format Output

| Style | Syntax | Example |
|---|---|---|
| f-strings (modern, preferred) | f'{variable}' | f'pi = {pi:.2f}' |
| .format() method | '{}'.format(x) | '{n} colors'.format(n=3) |
| % operator (C-style, legacy) | '%s' % (x) | 'Hi %s' % ('there') |

*f-strings support the same :.2f style precision specifiers used inside .format().*

INPUT & OUTPUT

# f-strings — Formatting Numbers Cleanly

The :.2f inside { } means 'show 2 digits after the decimal point'.

```
pi = 3.1415926
e = 2.7182818
print(f'Values: pi={pi} and e={e}')

values = [pi, e]
print(f'pi={values[0]:.2f}, e={values[1]:.3f}')
```

OUTPUT

```
Values: pi=3.1415926 and e=2.7182818

pi=3.14, e=2.718
```

INPUT & OUTPUT

# print() — Useful Options

print() accepts multiple values (comma-separated) and keyword options like sep and end.

```
print("Addition 11+5 :", 11 + 5)

print()                    # prints a blank line

# Single-line conditional expression
x = 11
print("Even") if x % 2 == 0 else print("Odd")
```

OUTPUT

```
Addition 11+5 : 16

Odd
```

INPUT & OUTPUT

# Reading & Writing Files — File Modes

➔ 'r' — read only. Raises IOError if the file does not exist

➔ 'w' — write only. Overwrites the file if it exists, else creates it

➔ 'a' — append. Adds to the end without erasing existing content

➔ 'r+' / 'w+' — read AND write in the same file handle

➔ Always prefer the 'with open(...) as f:' pattern — it closes the file automatically

INPUT & OUTPUT

# Writing to a File

'w' mode creates the file if needed, or overwrites it completely if it already exists.

```
with open('Test1.txt', 'w') as wf:
    wf.write("This is a sample test line")
    wf.write("\nThis is a next line. ")
    wf.write("Continuing last line.")
    print('Test1.txt created')
```

OUTPUT

```
Test1.txt created
```

INPUT & OUTPUT

# Reading a File — read(), readlines(), readline()

read() returns the whole file as one string; readlines() returns a list of lines.

```
with open('Test1.txt', 'r') as f:
    print(f.read())         # entire file as one string

with open('Test1.txt', 'a') as f:
    f.write(" Appended text.")

with open('Test1.txt', 'r') as f:
    print(f.readlines())    # list of lines
```

OUTPUT

```
This is a sample test line
This is a next line.
Continuing last line.

['This is a sample test line\n',
 'This is a next line. ...']
```

INPUT & OUTPUT

# Moving Around a File — seek() & tell()

seek(offset) jumps the file pointer to a byte position; tell() reports the current position.

```
with open('Test1.txt', 'r') as f:
    print(f.readline())          # read first line
    print(f.read(23))            # next 23 characters
    f.seek(0)                     # jump back to start
    print("Position:", f.tell())

print("Name:", f.name)
print("Mode:", f.mode)
```

OUTPUT

```
This is a sample test line

This is a next line. Con

Position: 0

Name: Test1.txt
Mode: r
```

04

# Python Built-in Functions

➔ Ready-to-use functions — no import needed

➔ join, pow, abs, round, divmod, zip, map, filter, reduce, all/any

BUILT-IN FUNCTIONS

# Today's Built-in Functions at a Glance

| Function | Purpose |
|---|---|
| join() | Combine a list of strings into one string, using a separator |
| pow() | Power / modular exponentiation: x**y or (x**y) % z |
| abs() | Absolute value of a number |
| round() | Round a number to a given number of decimal places |
| divmod() | Return (quotient, remainder) of a division in one call |
| zip() | Pair up elements from two or more iterables |
| map() | Apply a function to every element of an iterable |
| filter() | Keep only the elements that satisfy a condition |
| reduce() | Combine all elements into a single cumulative value |
| all() / any() | Check whether all / any values in an iterable are True |

BUILT-IN FUNCTIONS

# join() — Opposite of split()

The separator string calls .join() on a list of strings.

```
s = "This is a sample text"
words = s.split()               # ['This','is','a',...]

print('-'.join(words))
print(' | '.join(words))

# using lambda
x = lambda a: ' '.join(a)
print(x(words))
```

OUTPUT

```
This-is-a-sample-text

This | is | a | sample | text

This is a sample text
```

BUILT-IN FUNCTIONS

# pow(), abs(), round()

pow(x, y, z) computes (x**y) % z efficiently — useful for large numbers.

```
print("2**5:", 2**5, " pow(2,5):", pow(2, 5))
print("(2**5)%3:", pow(2, 5, 3))

print("abs(-57):", abs(-57))
print("abs(12+2j):", abs(12 + 2j))   # magnitude

print("round(3.14,1):", round(3.14, 1))
print("round(2379,-2):", round(2379, -2))
```

OUTPUT

```
2**5: 32  pow(2,5): 32
(2**5)%3: 2

abs(-57): 57
abs(12+2j): 12.166...

round(3.14,1): 3.1
round(2379,-2): 2400
```

BUILT-IN FUNCTIONS

# divmod() — Quotient and Remainder Together

divmod(a, b) is equivalent to (a // b, a % b), returned as a single tuple.

```
a, b = 5, 2
print("a//b:", a // b)     # quotient
print("a%b:", a % b)       # remainder

quot, rem = divmod(a, b)
print("divmod():", quot, rem)
```

OUTPUT

```
a//b: 2
a%b: 1

divmod(): 2 1
```

BUILT-IN FUNCTIONS

# zip() — Pairing Iterables Together

zip() stops at the shortest iterable. Use the * operator to 'unzip'.

```
y = list(zip([1, 2, 3], ('abc', 'pqr', 'xyz')))
print("zipped:", y)

a, b = zip(*y)              # unzip!
print("a:", a)
print("b:", b)

matrix = [[1,2,3],[4,5,6],[7,8,9]]
transpose = [list(r) for r in zip(*matrix)]
print("transpose:", transpose)
```

OUTPUT

```
zipped: [(1,'abc'),(2,'pqr'),(3,'xyz')]

a: (1, 2, 3)
b: ('abc', 'pqr', 'xyz')

transpose: [[1,4,7],[2,5,8],[3,6,9]]
```

BUILT-IN FUNCTIONS

# map() — Apply a Function to Every Element

map(function, iterable) returns a map object — wrap it in list() to see the results.

```
a = [1, 2, 3, 4, 5]

def add_two(x):
    return x + 2

print(list(map(add_two, a)))

# same thing with lambda - very common combo
print(list(map(lambda x: x + 2, a)))
```

OUTPUT

```
[3, 4, 5, 6, 7]

[3, 4, 5, 6, 7]
```

BUILT-IN FUNCTIONS

# filter() — Keep Only What Passes the Test

filter(function, iterable) keeps elements where function(element) is truthy.

```
def div_3(x):
    return x % 3 == 0

a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
print(list(filter(div_3, a)))

words = ['hello','are','cat','hi','heart']
print(list(filter(lambda w: w[0]=='h', words)))
```

OUTPUT

```
[3, 6, 9]

['hello', 'hi', 'heart']
```

BUILT-IN FUNCTIONS

# reduce() — Roll a Sequence Into One Value

reduce() lives in functools. It repeatedly applies function(accumulator, next_item).

```
from functools import reduce

arr = [77, 41, 42, 13]
print(reduce(lambda x, y: x + y, arr))   # like sum()

def digits_to_num(digits):
    return reduce(lambda x, y: x*10 + y, digits)

print(digits_to_num([3, 4, 3, 2, 1]))
```

OUTPUT

```
173

34321
```

BUILT-IN FUNCTIONS

# all() and any()

all() needs every value to be True; any() needs just one.

```
print(all([True, False, False]))
print(any([True, False, False]))

marks = [72, 65, 88, 91]
print("Everyone passed 40:", all(m >= 40 for m in marks))
print("Someone scored 90+:", any(m >= 90 for m in marks))
```

OUTPUT

```
False
True

Everyone passed 40: True
Someone scored 90+: True
```

# Recap — Choosing the Right Built-in

- ➔ Need to combine strings? -> join()
- ➔ Need power / modular power? -> pow()
- ➔ Need magnitude, ignore sign? -> abs()
- ➔ Need quotient AND remainder together? -> divmod()
- ➔ Need to pair up two lists? -> zip()
- ➔ Need to transform every element? -> map()
- ➔ Need to keep only some elements? -> filter()
- ➔ Need one cumulative result from a sequence? -> reduce()
- ➔ Need a yes/no check across a whole sequence? -> all() / any()